Machine Learning Practical

Practical No. 1

Title: Implementing K-Nearest Neighbor (KNN) Algorithm

Problem: Classify handwritten digits using the K-Nearest Neighbor (KNN) algorithm.

Solution:

Import necessary libraries:

```
import numpy as np

from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report
```

Load the digits dataset:

```
digits = load_digits()
```

Split the dataset into training and testing sets:

```
X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.2,
random_state=42)
```

Create a K-Nearest Neighbors classifier:

```
knn = KNeighborsClassifier(n_neighbors=3)
```

Train the classifier:

```
knn.fit(X_train, y_train)
```

Predict the labels for the test set:

```
y_pred = knn.predict(X_test)
```

Calculate the accuracy of the classifier:

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy}")
```

Print the classification report:

```
print("Classification Report:")

print(classification_report(y_test, y_pred))
```

Expected Outcome:

```
Accuracy: 0.9833333333333333


Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        33
           1       0.97      1.00      0.98        28
           2       1.00      1.00      1.00        33
           3       0.97      1.00      0.99        34
           4       0.98      1.00      0.99        46
           5       0.98      0.98      0.98        47
           6       0.97      1.00      0.99        35
           7       1.00      0.97      0.99        34
           8       1.00      0.97      0.98        30
           9       0.97      0.93      0.95        40

    accuracy                           0.98       360
   macro avg       0.98      0.98      0.98       360
weighted avg       0.98      0.98      0.98       360
```

Practical No. 2

Title: Building a Decision Tree Model using the ID3 Algorithm

Problem: Predict whether a customer will churn or not using a decision tree model.

Solution:

Import necessary libraries:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, classification_report
```

Sample data for customer churn prediction:

```
data = {

    'Age': [25, 45, 35, 50, 23, 34, 40, 60, 48, 33],

    'Salary': [50000, 100000, 75000, 120000, 45000, 80000, 95000, 130000, 110000, 70000],

    'Churn': [0, 1, 0, 1, 0, 0, 1, 1, 1, 0]
```

}

Create a DataFrame:

df = pd.DataFrame(data)

Split the dataset into features and target variable:

X = df.drop('Churn', axis=1)

y = df['Churn']

Split the dataset into training and testing sets:

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Create a Decision Tree classifier using the ID3 algorithm:

clf = DecisionTreeClassifier(criterion='entropy')

Train the classifier:

clf.fit(X_train, y_train)

Predict the labels for the test set:

y_pred = clf.predict(X_test)

Calculate the accuracy of the classifier:

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy}")

Print the classification report:

print("Classification Report:")

print(classification_report(y_test, y_pred))

Expected Outcome:

```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           1       1.00      1.00      1.00         2

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```

Practical No. 3

Title: SVM Model

Problem: Developing a Support Vector Machine (SVM) model (e.g. to classify email messages as spam or not spam).

Solution:

Import necessary libraries:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import GridSearchCV

from sklearn import svm
```

Data Loading:

```
data = pd.read_csv("C:/Users/shalinik1/OneDrive - KPMG/Desktop/colleges/spam.csv")
```

Basic Eda:

```
data.head()

data.info()

data.describe()
```

Assigning Input & Output Parameters:

```
X = data['Email'].values

y = data['Label'].values

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=0)
```

Converting String to Integer:

```
cv = CountVectorizer()

X_train = cv.fit_transform(X_train)

X_test = cv.transform(X_test)

print(X_train)
```

print(X_test)

Model Fitting

from sklearn.svm import SVC

classifier = SVC(kernel = 'rbf', random_state = 10)

classifier.fit(X_train, y_train)

print(classifier.score(X_test,y_test))

from sklearn.metrics import accuracy_score

Make predictions

y_pred = classifier.predict(X_test)

Calculate accuracy

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy}")

Expected Outcome:

Basic EDA:

In [9]:
```
data.head()
```

Out[9]:

	Label	Email
0	ham	U reach orchard already? U wan 2 go buy ticket...
1	ham	I am real, baby! I want to bring out your inne...
2	ham	No da if you run that it activate the full ver...
3	ham	\AH POOR BABY!HOPE URFEELING BETTERSN LUV! PRO...
4	ham	Stop the story. I've told him i've returned it...

In [10]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 207 entries, 0 to 206
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Label   207 non-null    object
 1   Email   207 non-null    object
dtypes: object(2)
memory usage: 3.4+ KB
```

In [11]:
```
data.describe()
```

Out[11]:

	Label	Email
count	207	207
unique	2	207
top	ham	U reach orchard already? U wan 2 go buy ticket...
freq	179	1

Visualizing the data in X_train &X_test:

```
print(X_train)
```

```
  (0, 352)      1
  (0, 857)      1
  (0, 104)      1
  (0, 392)      1
  (0, 979)      1
  (0, 365)      1
  (0, 1050)     1
  (1, 586)      1
  (1, 891)      1
  (1, 153)      2
  (1, 637)      1
  (1, 400)      1
  (1, 761)      1
  (1, 303)      2
```

```
print(X_test)
```

```
  (0, 102)      1
  (0, 190)      1
  (0, 384)      1
  (0, 410)      1
  (0, 446)      1
  (0, 532)      1
  (0, 609)      1
  (0, 643)      1
  (0, 660)      1
  (0, 838)      1
  (0, 885)      1
  (0, 939)      1
  (0, 1005)     1
  (0, 1049)     1
```

Classifier Score:

```
0.8809523809523809
```

Accuracy score:

```
Accuracy: 0.8809523809523809
```

Practical No. 4

Title: Naïve Bayes classifier

Problem: Building a Naïve Bayes classifier (e.g. to classify movie reviews as positive or negative sentiment)

Solution:

Step 1: Import Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score

import nltk

from nltk.corpus import stopwords
```

Step 2: Load and Prepare Data

```
data = pd.read_csv('C:/Users/shalinik1/OneDrive - KPMG/Desktop/colleges/datasets/IMDB.csv')

print(data)
```

Expected Output:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive
...	...	...
49995	I thought this movie did a down right good job...	positive
49996	Bad plot, bad dialogue, bad acting, idiotic di...	negative
49997	I am a Catholic taught in parochial elementary...	negative
49998	I'm going to have to disagree with the previou...	negative
49999	No one expects the Star Trek movies to be high...	negative

data["review"][1]

```
'A wonderful little production. <br /><br />The filming technique is very unassuming- very old-time-BBC fashion and gives a com
forting, and sometimes discomforting, sense of realism to the entire piece. <br /><br />The actors are extremely well chosen- M
ichael Sheen not only "has got all the polari" but he has all the voices down pat too! You can truly see the seamless editing g
uided by the references to Williams\' diary entries, not only is it well worth the watching but it is a terrificly written and
performed piece. A masterful production about one of the great master\'s of comedy and his life. <br /><br />The realism really
comes home with the little things: the fantasy of the guard which, rather than use the traditional \'dream\' techniques remains
solid then disappears. It plays on our knowledge and our senses, particularly with the scenes concerning Orton and Halliwell an
d the sets (particularly of their flat with Halliwell\'s murals decorating every surface) are terribly well done.'
```

Step 3: Preprocess Data

import string

from nltk.corpus import stopwords

data["clean"]=data["review"].str.lower()

data["clean"][1]

```
'a wonderful little production. <br /><br />the filming technique is very unassuming- very old-time-bbc fashion and gives a com
forting, and sometimes discomforting, sense of realism to the entire piece. <br /><br />the actors are extremely well chosen- m
ichael sheen not only "has got all the polari" but he has all the voices down pat too! you can truly see the seamless editing g
uided by the references to williams\' diary entries, not only is it well worth the watching but it is a terrificly written and
performed piece. a masterful production about one of the great master\'s of comedy and his life. <br /><br />the realism really
comes home with the little things: the fantasy of the guard which, rather than use the traditional \'dream\' techniques remains
solid then disappears. it plays on our knowledge and our senses, particularly with the scenes concerning orton and halliwell an
d the sets (particularly of their flat with halliwell\'s murals decorating every surface) are terribly well done.'
```

data['cleandata']=data['review'].str.replace('[^a-z\']'," ")

data['cleandata']

print(data)

	review	sentiment	clean	cleandata
0	One of the other reviewers has mentioned that ...	positive	one of the other reviewers has mentioned that ...	One of the other reviewers has mentioned that ...
1	A wonderful little production. The...	positive	a wonderful little production. the...	A wonderful little production. The...
2	I thought this was a wonderful way to spend ti...	positive	i thought this was a wonderful way to spend ti...	I thought this was a wonderful way to spend ti...
3	Basically there's a family where a little boy ...	negative	basically there's a family where a little boy ...	Basically there's a family where a little boy ...
4	Petter Mattei's "Love in the Time of Money" is...	positive	petter mattei's "love in the time of money" is...	Petter Mattei's "Love in the Time of Money" is...
...	...	...	...	...
49995	I thought this movie did a down right good job...	positive	i thought this movie did a down right good job...	I thought this movie did a down right good job...
49996	Bad plot, bad dialogue, bad acting, idiotic di...	negative	bad plot, bad dialogue, bad acting, idiotic di...	Bad plot, bad dialogue, bad acting, idiotic di...
49997	I am a Catholic taught in parochial elementary...	negative	i am a catholic taught in parochial elementary...	I am a Catholic taught in parochial elementary...
49998	I'm going to have to disagree with the previou...	negative	i'm going to have to disagree with the previou...	I'm going to have to disagree with the previou...
49999	No one expects the Star Trek movies to be high...	negative	no one expects the star trek movies to be high...	No one expects the Star Trek movies to be high...

50000 rows × 4 columns

from sklearn.preprocessing import LabelEncoder
reviews = data['cleandata'].values

```
labels = data['sentiment'].values
encoder = LabelEncoder()
encoded_labels = encoder.fit_transform(labels)
encoded_labels
```

```
array([1, 1, 1, ..., 0, 0, 0])
```

Step 4: Split Data
Split the data into training and testing sets.

```
from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report

from sklearn.metrics import accuracy_score
X_train, X_test, y_train, y_test = train_test_split(data['cleandata'], data['sentiment
'], test_size=0.2, random_state=42)
```

Step 5: Vectorize Text Data
Convert the text data into numerical data using CountVectorizer.

```
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

Step 6: Train the Naïve Bayes Classifier

```
nb_classifier = MultinomialNB()
nb_classifier.fit(X_train_vec, y_train)
```

Step 7: Evaluate the Model
Evaluate the model's performance on the test data.

```
y_pred = nb_classifier.predict(X_test_vec)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

Expected Output:

```
Accuracy: 86.19%
```

Step 8: Make Predictions

```
new_review = ["This movie was fantastic!","Worst picture","ok we can see for onc
e"]
new_review_vec = vectorizer.transform(new_review)
prediction = nb_classifier.predict(new_review_vec)
print(f'Prediction: {prediction[0]}')
print(f'Prediction: {prediction[1]}')
print(f'Prediction: {prediction[2]}')
```

Expected Output:

```
Prediction: positive
Prediction: negative
Prediction: negative
```

Practical No. 5

Title: linear regression

Problem: Implementing linear regression (e.g. to predict housing prices based on features such as size and location).

Solution:

Step 1: Import Libraries

```
import numpy as np

import pandas as pd

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error
```

Step 2: Load and Prepare Data

```
df = pd.read_csv('C:/Users/shalinik1/OneDrive - KPMG/Desktop/colleges/dayanand sagar/FDP_ML/mlr.csv')
```

```
df
```

	Square_Feet	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	Neighborhood_Quality	House_Price	Location
0	1360	2	1	1981	0.599637	0	5	2.623829e+05	mumbai
1	4272	3	3	2016	4.753014	1	6	9.852609e+05	pune
2	3592	1	2	2016	3.634823	0	9	7.779774e+05	delhi
3	966	1	2	1977	2.730667	1	8	2.296989e+05	chennai
4	4926	2	1	1993	4.699073	0	8	1.041741e+06	bangalore
...	...	...	...	...	...	...	...	...	...
995	3261	4	1	1978	2.165110	2	10	7.014940e+05	bangalore
996	3179	1	2	1999	2.977123	1	10	6.837232e+05	chennai
997	2606	4	2	1962	4.055067	0	2	5.720240e+05	bangalore
998	4723	5	2	1950	1.930921	0	7	9.648653e+05	mumbai
999	3268	4	2	1983	3.108790	2	2	7.425993e+05	chennai

```
# Location mapping

location_mapping = {'chennai': 1, 'bangalore': 2, 'pune': 3, 'mumbai': 4, 'delhi': 5}

# Ensure all location names are strings and map them to numerical values

df['Location'] = [location_mapping[str(loc).lower()] for loc in df['Location']]

df
```

	Square_Feet	Num_Bedrooms	Num_Bathrooms	Year_Built	Lot_Size	Garage_Size	Neighborhood_Quality	House_Price	Location
0	1360	2	1	1981	0.599637	0	5	2.623829e+05	4
1	4272	3	3	2016	4.753014	1	6	9.852609e+05	3
2	3592	1	2	2016	3.634823	0	9	7.779774e+05	5
3	966	1	2	1977	2.730667	1	8	2.296989e+05	1
4	4926	2	1	1993	4.699073	0	8	1.041741e+06	2
...	...	...	...	...	...	...	...	...	...
995	3261	4	1	1978	2.165110	2	10	7.014940e+05	2
996	3179	1	2	1999	2.977123	1	10	6.837232e+05	1
997	2606	4	2	1962	4.055067	0	2	5.720240e+05	2
998	4723	5	2	1950	1.930921	0	7	9.648653e+05	4
999	3268	4	2	1983	3.108790	2	2	7.425993e+05	1

1000 rows × 9 columns

```
# Features and target variable

X = df[['Location', 'Lot_Size']]
```

```
y = df['House_Price']
```

Step 3: Split Data

```
# Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Step 4: Train the model

```
# Create and train the linear regression model

linear_regressor = LinearRegression()

linear_regressor.fit(X_train, y_train)
```

Step 5: Testing the model

```
# Predict house prices

y_pred = linear_regressor.predict(X_test)
```

Step 6: Model Evaluation

```
# Calculate mean squared error

mse = mean_squared_error(y_test, y_pred)

print("Multiple Linear Regression - House Prices Prediction")

print("Mean Squared Error:", mse)

print("Predicted Prices:", y_pred)

print("Actual Prices:", y_test.values)
```

Expected Output:

```
Multiple Linear Regression - House Prices Prediction
Mean Squared Error: 66239910076.95545
Predicted Prices: [597609.16264576 560127.94435746 590803.08500292 673984.06245424
 674310.94407548 615560.01638958 569576.72250188 561536.85063089
 643132.6988332  694046.62633716 653450.6089392  601601.89627649
 621251.50058507 699579.40857677 561989.99859077 590961.92704384
 533272.40807325 578057.92274267 641899.86264051 645851.80221549
 662530.86822755 549719.24554544 536111.59177924 672037.18034114
 534086.77498654 649288.48859032 684578.76004082 685650.66176496
 543797.18733594 663592.77181809 600952.69277059 674317.53749664
 581241.20637832 683654.55369552 696713.56106591 698046.83048932
 655424.2795464  665998.40057597 675159.6573579  600932.25271009
 602968.77604611 644516.14673016 654608.27844136 620500.93115522
 657985.76891603 710678.55769134 541991.53390003 577688.38276927
 576722.78079575 563405.66001051 685642.57004629 565659.86691539
 601138.88206236 688563.68821369 646692.89625104 627913.18991798
 623088.3370171  638697.36691303 639813.31335994 550137.54714671
 573544.22682754 598764.32955775 695289.01887619 694262.04551822
 544180.17190806 661001.13328548 669228.28582826 659531.75840615
 549583.63820886 666340.93847089 559570.45488386 683804.1709158
 613720.05697906 570144.25177053 690886.05796661 588566.23357506
 658861.64480219 575178.5715056  544927.52181527 561702.93419724
 596118.91260609 671604.81266837 597402.34584537 605558.63195847
 711789.59736475 691409.40530479 581550.48572724 699218.87374144
 688204.77584571 616720.42163056 582192.88847194 639154.5907419
 549226.99696171 604724.053478   555973.79038919 665518.7895951
 569276.99101155 645433.23518459 686540.42905919 527169.15583753
```

```
 Actual Prices: [ 901000.4882  494537.5115  949404.2007 1040389.048
   724033.5605  998439.2371  909713.435   792681.5184  947490.7775
   774150.0468  654466.9454  217462.4451 1034380.204   530476.1233
   194353.7361  441315.0885  354744.6306  755402.3727  348893.554
   763645.6102  475767.3505  676108.8512  887527.9072  438250.1762
   402213.4219  472657.6143  453579.1055  191907.6525  489595.4125
   270230.6377  893926.6295  771922.8498  338350.5954  457643.098
   775773.2238  474659.8601  570399.7428  280961.6804  147601.5516
   300656.0791  307101.6858  486872.0567  304653.636  1092026.897
  1008539.156   747938.7115  818613.7108 1028282.312   694917.3009
   261227.3187  762769.5056  813296.809   705754.432   530824.1968
   809686.2011  190828.1399  335332.5928  728289.8873  217213.6452
   993273.0797  377511.2685  322312.8625  517515.406   604936.8305
   551324.4887  468699.4413  729494.9062  951093.547   746167.7281
   480961.442   875981.0269  264215.3398  574912.7662  684425.2003
   733612.8164  551444.6316  188531.4235  537619.5908  872676.4342
   545482.742   397346.1109  534533.02    191117.1012  149743.5
   379005.7119  964865.2986  935451.9336  552158.5008  224507.2001
   934919.3828  781388.9357  364834.6822  798362.775   706686.3209
  1068538.106   760720.1465  898283.8867  863289.3969  605369.8615
   917235.4105  871566.5627  262707.2789  605143.9591  470083.2904
   913774.389   560304.6612  198773.4757  285546.5017 1037911.652
```